



G3 My First Game

Overview

- Reversing
- Disassembler
- Windows APIs
- Let's play game!!
- Disassemble
- Game Reversing
- PoC. 1
- WinDbg: edit
- PoC. 2

Reversing

- 리버싱

- e.g.) main.c는 Windows에서 main.exe로 변환되어 실행가능한 파일이 되어 실행함.
- 모든 실행파일은 Windows에서 .exe 형태임. (Linux는 elf 포맷, 확장자는 없음)
- 그럼, 내부 실행은 어떻게 동작하는가?
- 개발자가 작성한 코드가 순차적으로 실행되며 동작함.
- 그럼 그 명령어들은 어떻게 남아있나?
- Windows와 Linux는 다를지 모르지만, Linux 기준으로는 .text 영역에 코드가 바이너리 형태로 남음.
- 그것은 순수 기계어(01111001)임. 알아볼 수 없음.
- 다만, 그 기계어는 어셈블리 언어와 1:1 대응.
- 그래서 실행파일을 디버깅하면 어셈블리 코드를 보며 흐름을 알 수 있음.
- 그러면 어셈블리 코드를 디스어셈블하면 슈도코드 형태로도 복원이 가능하겠음을 어렵짐작 할 수 있음.
- 다만, 어셈블리 코드는 개발자가 작성한 코드를 읽어 컴파일러가 생성함.
- 컴파일러가 요즘 너무 똑똑해서 최적화를 좀 함. 그래서 슈도코드 형태로 복원된 코드는 동작이 다소 난해하게 느껴질 수 있음.
- 심볼(변수명, 함수명) 정보가 남지않으면 리버싱 과정은 매우 난해하다고 할 수 있음. → 구조체가 있는 경우...

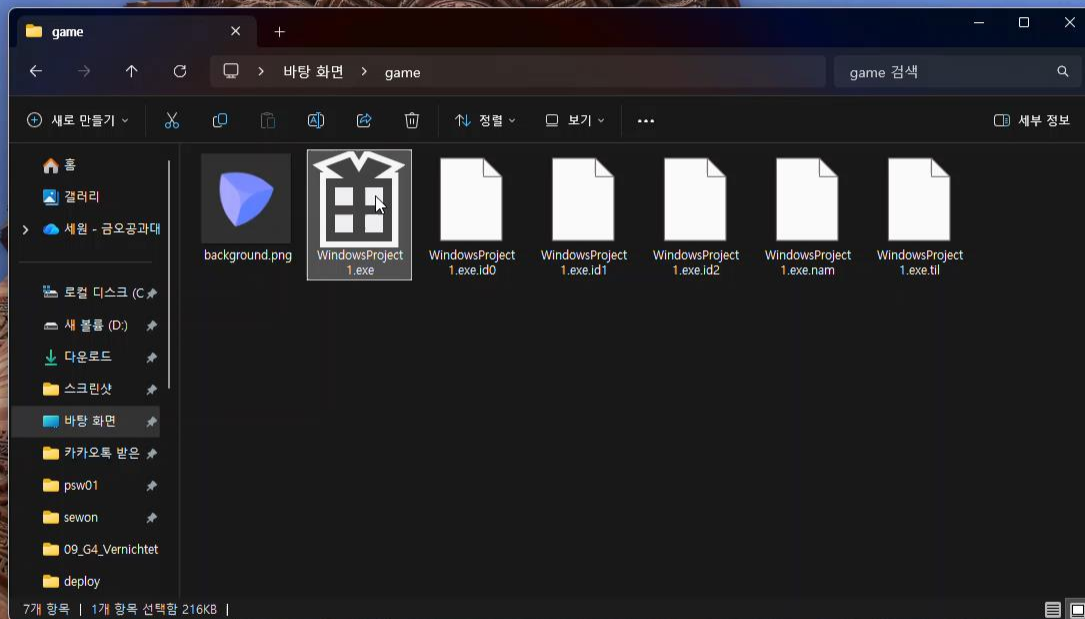
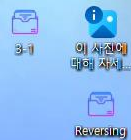
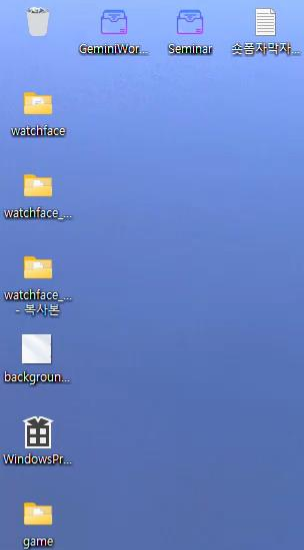
Disassembler

- IDA, Ghidra, Binary ninja, etc.
 - 디스어셈블을 지원하는 도구들이 좀 있음.
 - IDA는 정석적으로 사용하려면 돈백만원 듬.
 - Ghidra는 완전 오픈소스임.
 - Binary ninja가 써본적 없지만, 그나마 합리적일 것 같음.
 - 필자는 IDA를 사용.
 - 디스어셈블러는 컴파일러가 최적화해둔 어셈블리를 이용해 생성된 기계어를 디스어셈블해 슈도코드로 복원하기 때문에
→ 디스어셈블러 도구에 따라 슈도코드 결과가 다를 수 있음.
 - 동작은 동일하지만, 좀 더 가독성 있는 슈도코드가 생성될 수도 있다는 말임.

*동작은 같지만, 정말 골때리는 형태의 코드가 나올 수 있으니 다른 도구로 디스어셈블 하기도 함.
e.g.) goto - LABEL*

Windows APIs

- Win32 API라고도 함.
 - Win32 API와 일반 DLL과 차이가 있다면,
 - Windows API (Win API): Win 기본 제공 syscall들, 창을 띄우고(user32.dll) 파일을 읽고(kernel32.dll) 그림을 그리는(gdi32.dll) 등
 - 확장 DLL은 좀 다름.
 - 확장 DLL (e.g. d3dx9_43.dll): Win 핵심 기능 ✕ (MSL나 다른 개발사가 추가 확장한 기능)
 - - d3dx9_43.dll: 게임 환경을 위한 DirectX 9 확장 라이브러리임.
- 본 리버싱 대상 .exe는 확장 DLL 사용: d3dx9_43.dll

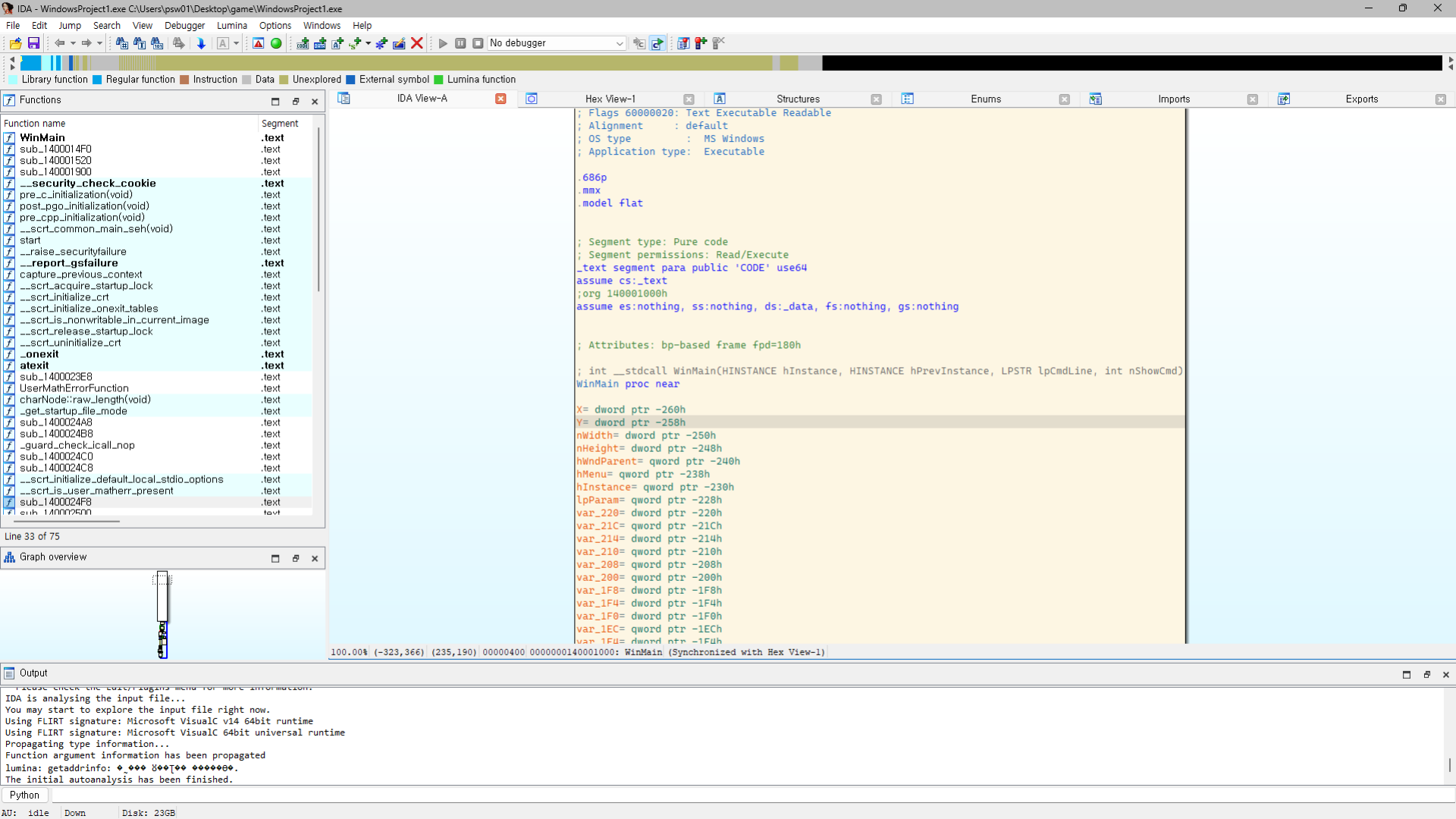


Let's play game!!

- Analysis
 - 마우스 입력을 받고 화면이 회전함.
 - 정육면체 내부에 갇힌 형태
 - 키보드 입력은 시도 안 해봄
 - 마우스 클릭으로 보라색을 터트릴 수 있나 싶어서 시도해보니 안됨. (클릭은 받지 않음)
 - 마우스를 위 아래로 돌리면 시점이 초기화 되는 느낌이 듦.

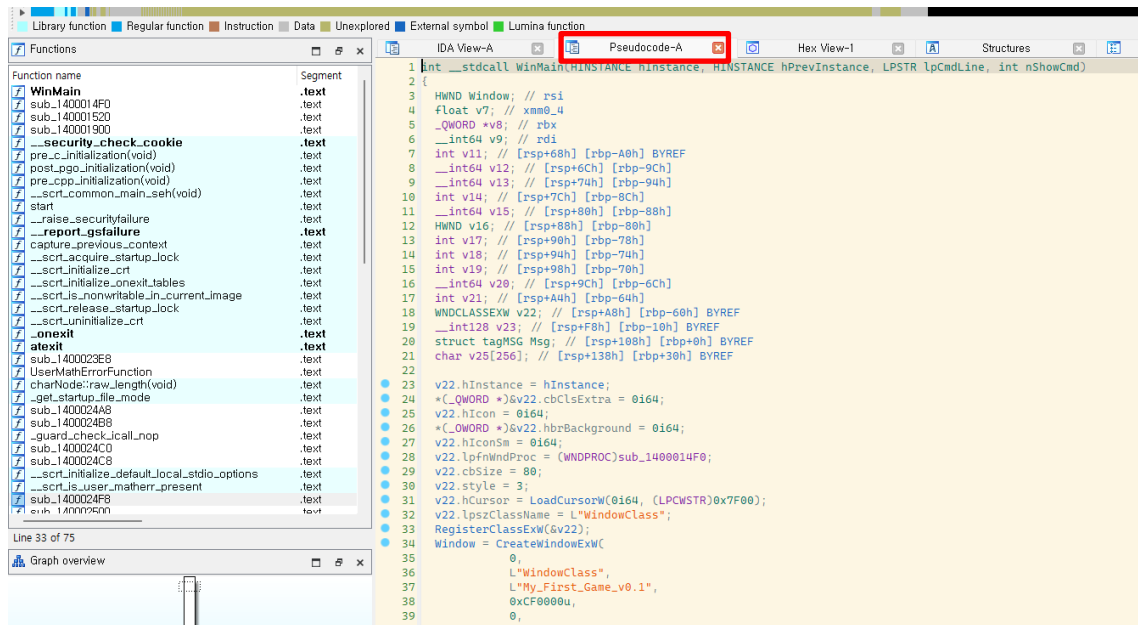
목표: 게임 내부 어딘가에 FLAG가 있고 그 FLAG를 찾아 제출하는 것

어디 있을지는 찾아야 함. 게임 코드 내부에 있을 수 있고, 특정 조건이 만족해야 할 수도 있고, 숨겨져 있을 수도 있고...



Disassemble

- 디스어셈블하면
- Functions(함수들), graph(분기에 따른 흐름을 볼 수 있음), IDA-view가 보임.
- IDA-view: 기계어를 어셈블리 언어와 1:1 매핑한 결과를 보임.
- → IDA-view에서 tab.
- 슈도코드를 보여줌.



Disassemble

- 원래 저런 문제는 리눅스 ELF 포맷 파일이 대부분이었음.
- 근데 이번에는 .exe 포맷이라 들고옴(게임 형태이기도 하고).
- 일반적으로 함수는 `int main(void)`
- 이번 문제는 `WinMain` 이 보임. 심볼도 잘 남아있고...
- WinMain이 뭔지 찾아봄.

- int main(void): console 창 기반 프로그램 사용 시 표준 진입점

- int WINAPI WinMain(...): GUI 기반 프로그램 사용 시 표준 진입점

→ WinOS: 창을 만들고 마우스/키보드 입력을 처리하기 위해 WinMain 구조를 요구함.

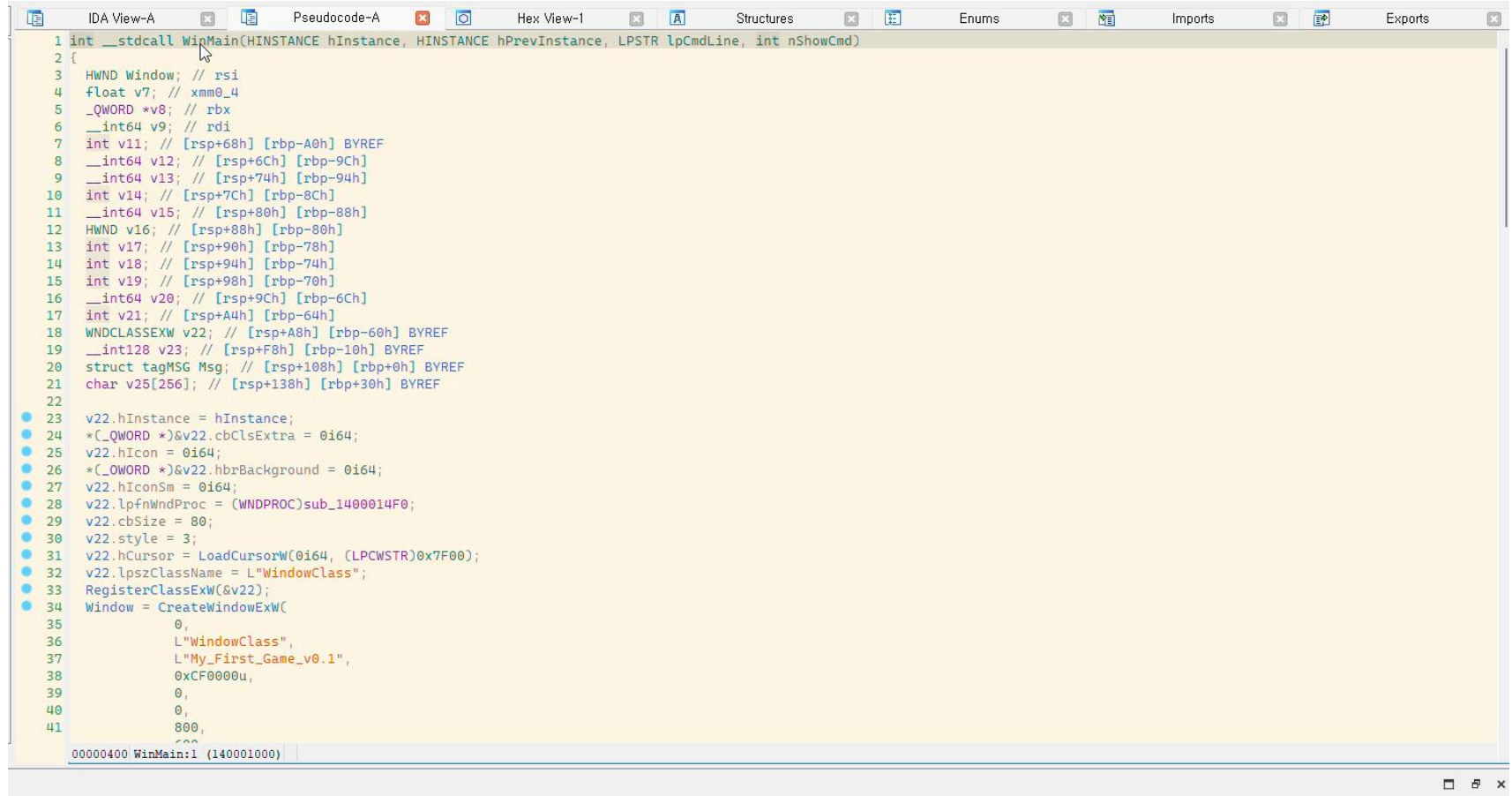
형태: int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int nCmdShow)

```
#include <windows.h>

int WINAPI WinMain(...) {
    HWND hwnd = CreateWindow(...);           // 게임 창 만들기(창 크기, 타이틀, etc.)
    ShowWindow(hwnd, nCmdShow);              // 게임 창 화면을 모니터에 띄우기

    MSG msg;
    while (true) {                           // 무한 루프
        if (PeekMessage(&msg, 0, 0, PM_REMOVE)) { // 키보드, 마우스 입력 처리
            TranslateMessage(&msg);
            DispatchMessage(&msg);
        }
        UpdateGame();                         // 이벤트에 대한 처리, 게임 로직 업데이트
        RenderGraphics();                     // 그래픽 갱신 (마우스 이동에 따른 화면 변경)
    }
    return 0;
}
```

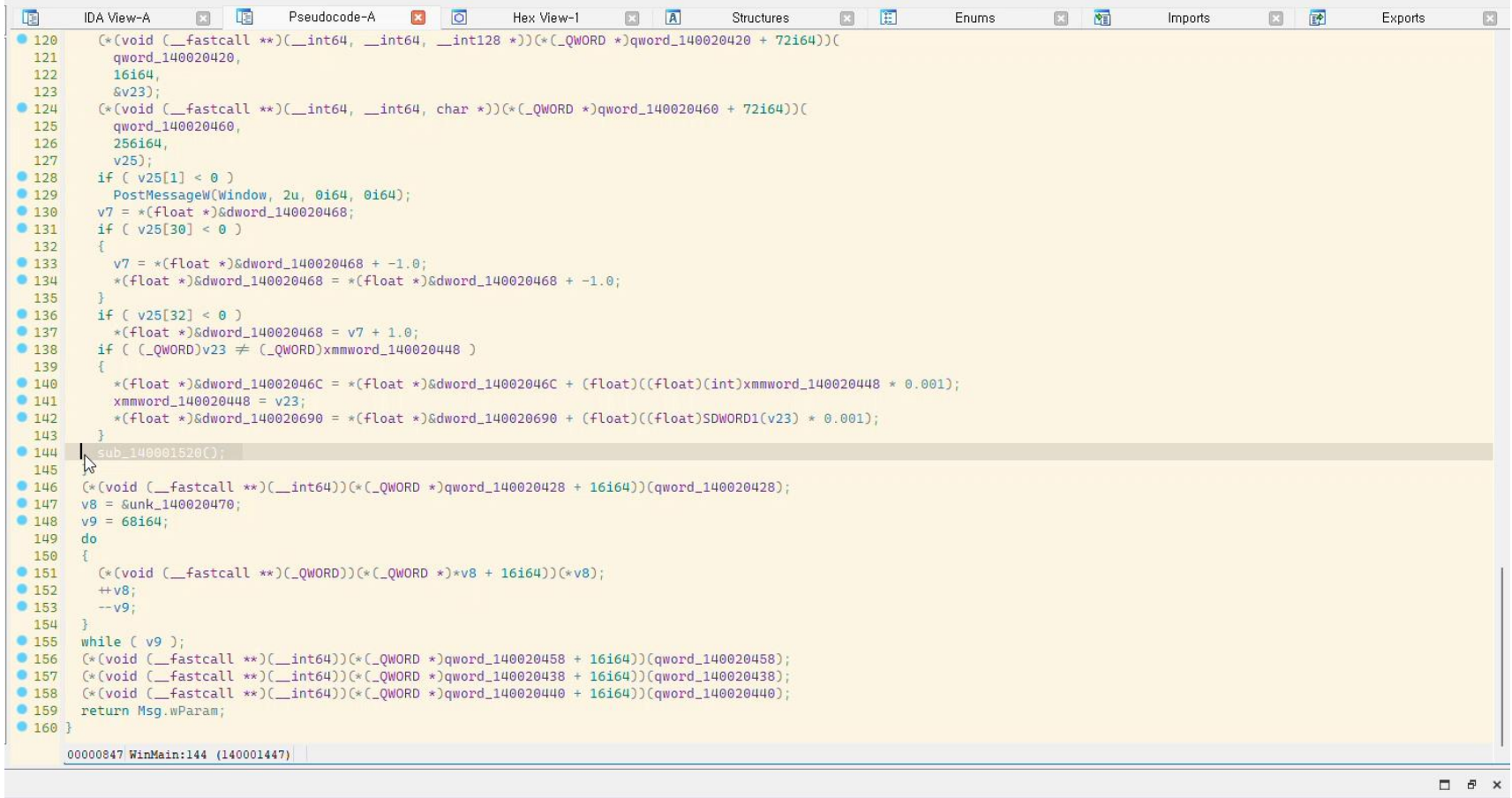
Game Reversing



```
1 int __stdcall WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int nShowCmd)
2 {
3     HWND Window; // rsi
4     float v7; // xmm0_4
5     _QWORD *v8; // rbx
6     __int64 v9; // rdi
7     int v11; // [rsp+68h] [rbp-A0h] BYREF
8     __int64 v12; // [rsp+6Ch] [rbp-9Ch]
9     __int64 v13; // [rsp+74h] [rbp-94h]
10    int v14; // [rsp+7Ch] [rbp-8Ch]
11    __int64 v15; // [rsp+80h] [rbp-88h]
12    HWND v16; // [rsp+88h] [rbp-80h]
13    int v17; // [rsp+90h] [rbp-78h]
14    int v18; // [rsp+94h] [rbp-74h]
15    int v19; // [rsp+98h] [rbp-70h]
16    __int64 v20; // [rsp+9Ch] [rbp-6Ch]
17    int v21; // [rsp+A4h] [rbp-64h]
18    WNDCLASSEXW v22; // [rsp+A8h] [rbp-60h] BYREF
19    __int128 v23; // [rsp+F8h] [rbp-10h] BYREF
20    struct tagMSG Msg; // [rsp+108h] [rbp+0h] BYREF
21    char v25[256]; // [rsp+138h] [rbp+30h] BYREF
22
23    v22.hInstance = hInstance;
24    *(_QWORD *)&v22.cbClsExtra = 0i64;
25    v22.hIcon = 0i64;
26    *(_QWORD *)&v22.hbrBackground = 0i64;
27    v22.hIconSm = 0i64;
28    v22.lpfnWndProc = (WNDPROC)sub_1400014F0;
29    v22.cbSize = 80;
30    v22.style = 3;
31    v22.hCursor = LoadCursorW(0i64, (LPCWSTR)0x7F00);
32    v22.lpszClassName = L"WindowClass";
33    RegisterClassExW(&v22);
34    Window = CreateWindowExW(
35        0,
36        L"WindowClass",
37        L"My_First_Game_v0.1",
38        0xCF0000u,
39        0,
40        0,
41        800,
```

00000400 WinMain:1 (140001000)

Game Reversing



The screenshot shows the IDA Pro interface with the 'Pseudocode-A' view selected. The code is a C-style function with various arithmetic and control flow operations. The function starts with a call to `qword_140020420` and `16i64`, followed by a call to `qword_140020460` and `256i64`. It then enters a loop structure with `if` and `while` statements, performing floating-point calculations on `v7` and `v9`. The function ends with a `return` statement and a `Msg.wParam` parameter.

```
120  (* (void (__fastcall *) (__int64, __int64, __int128 *)) (qword_140020420 + 72i64)) (
121      qword_140020420,
122      16i64,
123      &v23);
124  (* (void (__fastcall *) (__int64, __int64, char *)) (qword_140020460 + 72i64)) (
125      qword_140020460,
126      256i64,
127      v25);
128  if ( v25[1] < 0 )
129      PostMessageW(Window, 2u, 0i64, 0i64);
130  v7 = *(float *) &dword_140020468;
131  if ( v25[30] < 0 )
132  {
133      v7 = *(float *) &dword_140020468 + -1.0;
134      *(float *) &dword_140020468 = *(float *) &dword_140020468 + -1.0;
135  }
136  if ( v25[32] < 0 )
137      *(float *) &dword_140020468 = v7 + 1.0;
138  if ( (_QWORD)v23 != (_QWORD)xmmword_140020448 )
139  {
140      *(float *) &dword_14002046C = *(float *) &dword_14002046C + (float)((float)(int)xmmword_140020448 * 0.001);
141      xmmword_140020448 = v23;
142      *(float *) &dword_140020690 = *(float *) &dword_140020690 + (float)((float)SDWORD1(v23) * 0.001);
143  }
144  sub_140001520();
145
146  (* (void (__fastcall *) (__int64)) (qword_140020428 + 16i64)) (qword_140020428);
147  v8 = &unk_140020470;
148  v9 = 68i64;
149  do
150  {
151      (* (void (__fastcall *) (_QWORD)) (qword_140020458 + 16i64)) (v8);
152      ++v8;
153      --v9;
154  }
155  while ( v9 );
156  (* (void (__fastcall *) (__int64)) (qword_140020458 + 16i64)) (qword_140020458);
157  (* (void (__fastcall *) (__int64)) (qword_140020438 + 16i64)) (qword_140020438);
158  (* (void (__fastcall *) (__int64)) (qword_140020440 + 16i64)) (qword_140020440);
159  return Msg.wParam;
160 }
```

00000847 WinMain:144 (140001447)

Game Reversing

```
IDA View-A Pseudocode-A
33 RegisterClassExW(&v22);
34 Window = CreateWindowExW(
35     0,
36     L"WindowClass",
37     L"My_First_Game_v0.1",
38     0xCF0000u,
39     0,
40     0,
41     800,
42     600,
43     0i64,
44     0i64,
45     hInstance,
46     0i64);
47 ShowWindow(Window, nShowCmd);
48 v13 = 0i64;
49 qword_140020440 = (__int64)Direct3DCreate9(0x20u);
50 v14 = 0;
51 v15 = 1i64;
52 v20 = 0i64;
53 v21 = 0;
54 v17 = 1;
55 v16 = Window;
56 v11 = 800;
57 v12 = 0x16000000258i64;
58 v18 = 1;
59 v19 = 80;
60 (*(void (__fastcall *))(__int64, _QWORD, __int64, H
61     qword_140020440,
62     0i64,
63     1i64,
64     Window,
65     32,
66     &v11,
67     &qword_140020448);
68 sub_140001900();
```

```
109 while ( 1 ) // 무한 루프
110 {
111     while ( PeekMessageW(&Msg, 0i64, 0, 0, 1u) )
112     {
113         TranslateMessage(&Msg);
114         DispatchMessageW(&Msg);
115     }
116     if ( Msg.message == 18 )
117         break;
118     (*(void (__fastcall *))(__int64))(*(_QWORD *)qword_140020460 + 56
119     (*(void (__fastcall *))(__int64))(*(_QWORD *)qword_140020420 + 56
120     (*(void (__fastcall *))(__int64, __int64, __int128 *))(*(_QWORD *)
121         qword_140020420,
122         16i64,
123         &v23);
124     (*(void (__fastcall *))(__int64, __int64, char *))(*(_QWORD *)qwc
125         qword_140020460,
126         256i64,
127         v25);
128     if ( v25[1] < 0 )
129         PostMessageW(Window, 2u, 0i64, 0i64);
130     v7 = *(float *)&dword_140020468;
131     if ( v25[30] < 0 )
132     {
133         v7 = *(float *)&dword_140020468 + -1.0;
134         *(float *)&dword_140020468 = *(float *)&dword_140020468 + -1.0;
135     }
136     if ( v25[32] < 0 )
137         *(float *)&dword_140020468 = v7 + 1.0;
138     if ( (_QWORD)v23 != (_QWORD)xmmword_140020448 )
139     {
140         *(float *)&dword_14002046C = *(float *)&dword_14002046C + (floa
141         xmmword_140020448 = v23;
142         *(float *)&dword_140020690 = *(float *)&dword_140020690 + (floa
143     }
144     sub_140001520();
145 }
```

Game Reversing

- 해당 함수를 호출하는 dll에 대해 조사하고(d3dx9_43.dll)
- 해당 dll에서 호출되는 함수를 조사하다 보면..
- 정보가 채워짐.

```
83 void *v80; // [rsp+1C0h] [rbp+C0h] BYREF
84
85 D3DXCreateTextureFromFileW(qword_140020438, L"background.png", &qword_140020458);
86 v0 = 0;
87 v20 = xmmword_140003660;
```

```
84 // D3DXCreateTextureFromFileW(pDevice, pSrcFile, *ppTexture)
85 // - pDevice: 텍스처를 생성할 Direct3D 장치 인터페이스 포인터 [in]
86 // - pSrcFile: 로드할 이미지 파일 경로 [in]
87 // - ppTexture: 생성된 텍스처 인터페이스를 받을 주소 [out]
88 D3DXCreateTextureFromFileW(pDevice, L"background.png", &ppTexture);
89 size = 0;
90 v19 = xmmword_140003660;
```

Game Reversing

```
152 (* (v1 + 208))(qword_140020438, 384i64, 0i64);
153 (* (qword_140020428 + 88i64))(qword_140020428, 0i64, 0i64, &v80, 0);
154 v2 = v80;
155 v3 = v19;
156 v4 = 3i64;
157 do
158 {
159     v2 += 8;
160     v5 = *v3;
161     v6 = *(v3 + 1);
162     v3 += 32;
163     *(v2 - 8) = v5;
164     v7 = *(v3 - 6);
165     *(v2 - 7) = v6;
166     v8 = *(v3 - 5);
167     *(v2 - 6) = v7;
168     v9 = *(v3 - 4);
169     *(v2 - 5) = v8;
170     v10 = *(v3 - 3);
171     *(v2 - 4) = v9;
172     v11 = *(v3 - 2);
173     *(v2 - 3) = v10;
174     v12 = *(v3 - 1);
175     *(v2 - 2) = v11;
176     *(v2 - 1) = v12;
177     --v4;
178 }
179 while ( v4 );
```

```
155 (*(pDevice_ptr + 208))(pDevice, 384i64, 0i64); // IDirect3DDevice9::DrawIndexedPrimitive
156 (*(pDestRect
157     + 88i64))(
158     pDestRect, // IDirect3DSwapChain9::GetBackBuffer
159     0i64, // this ptr
160     0i64, // iBackBuffer: 0
161     &ppBackBuffer, // D3DBACKBUFFER_TYPE: 0
162     0); // IDirect3DSurface9: ppBackBuffer
163 ppBackBuffer_ = ppBackBuffer;
164 something_struct_ = something_struct;
165 v4 = 3i64;
166 do
167 {
168     ppBackBuffer_ += 8;
169     v5 = *something_struct_; // OWORD: 16byte
170     v6 = *(something_struct_ + 1);
171     something_struct_ += 32;
172     *(ppBackBuffer_ - 8) = v5;
173     v7 = *(something_struct_ - 6);
174     *(ppBackBuffer_ - 7) = v6;
175     v8 = *(something_struct_ - 5);
176     *(ppBackBuffer_ - 6) = v7;
177     v9 = *(something_struct_ - 4);
178     *(ppBackBuffer_ - 5) = v8;
179     v10 = *(something_struct_ - 3);
180     *(ppBackBuffer_ - 4) = v9;
181     v11 = *(something_struct_ - 2);
182     *(ppBackBuffer_ - 3) = v10;
183     v12 = *(something_struct_ - 1);
184     *(ppBackBuffer_ - 2) = v11;
185     *(ppBackBuffer_ - 1) = v12;
186     --v4;
187 }
188 while ( v4 );
```

```

180 (*(&qword_140020428 + 96i64))(q
181 v13 = &unk_140020470;
182 v14 = 0;
183 v15 = &unk_140020470;
184 v16 = 0i64;
185 v17 = 68i64;
186 do
187 {
188     D3DXCreateMeshFVF(&unk_1400
189 (*(&v13 + 120i64))(&v13, 0i6
190 memcpy(v80, &unk_1400068F0 +
191 (*(&v13 + 128i64))(&v13);
192 (*(&v13 + 136i64))(&v13, 0i6
193 memcpy(v80, &unk_14001AE10 +
194 result = (*(&v13 + 144i64))(&
195 v0 += &asc_140003410[v16];
196 ++v13;
197 v14 += &unk_140003520 + v16
198 v16 += 2i64;
199 v15 += 8;
200 --v17;
201 }
202 while ( v17 );
203 return result;
204 }

```

Pseudocode-A

```

.data:00000001400068D0 qword_1400068D0 dq 80000h
.data:00000001400068D8 qword_1400068D8 dq 2000000h
.data:00000001400068E0 dword_1400068E0 dd 1
.data:00000001400068E4 align 10h
.data:00000001400068F0 vertex_layout db 0
.data:00000001400068F1 db 0
.data:00000001400068F2 db 0
.data:00000001400068F3 db 0
.data:00000001400068F4 db 0FCh
.data:00000001400068F5 db 0DFh
.data:00000001400068F6 db 25h ; %
.data:00000001400068F7 db 3Fh ; ?
.data:00000001400068F8 db 0E9h
.data:00000001400068F9 db 7Fh ; 0
.data:00000001400068FA db 49h ; I
.data:00000001400068FB db 3Eh ; >
.data:00000001400068FC db 0FFh
.data:00000001400068FD db 0FFh
.data:00000001400068FE db 0FFh
.data:00000001400068FF db 0FFh
.data:0000000140006900 db 0
.data:0000000140006901 db 0
.data:0000000140006902 db 0
.data:0000000140006903 db 0
.data:0000000140006904 db 0
.data:0000000140006905 db 0
.data:0000000140006906 db 0
.data:0000000140006907 db 0
.data:0000000140006908 db 0
.data:0000000140006909 db 0
.data:000000014000690A db 0
.data:000000014000690B db 0
.data:000000014000690C db 2Ah ; *
.data:000000014000690D db 0
.data:000000014000690E db 0A6h
.data:000000014000690F db 3Dh ; =
.data:0000000140006910 db 0E9h
.data:0000000140006911 db 7Fh ; 0
.data:0000000140006912 db 49h ; I
.data:0000000140006913 db 3Eh ; >
.data:0000000140006914 db 0FFh
.data:0000000140006915 db 0FFh
.data:0000000140006916 db 0FFh
.data:0000000140006917 db 0
.data:0000000140006918 db 0
.data:0000000140006919 db 0
.data:000000014000691A db 0
.data:000000014000691B db 0
.data:000000014000691C db 0
.data:000000014000691D db 0
.data:000000014000691E db 0
000050F0 00000001400068F0: .data:vertex_layout (Synchronized with H

```

ng

```

.64))(pDestRect);
'0;

```

40020470;

```

{
+ table_idx * 2),
dx],

```

```

oon/gcc/mingw64/incl
))(&v13, 0i64, appBa
'er, &vertex_layout +
))(&v13,
))(&v13, 0i64, appBa
'er, &index_layout +
))(&v13,
able_idx];

```

```

index_buffer + table
;

```

Pseudocode-A

```

.data:000000014001AE0C db 0
.data:000000014001AE0D db 0
.data:000000014001AE0E db 0
.data:000000014001AE0F db 0
.data:000000014001AE10 index_layout db 16h
.data:000000014001AE11 db 0
.data:000000014001AE12 db 15h
.data:000000014001AE13 db 0
.data:000000014001AE14 db 0Ch
.data:000000014001AE15 db 0
.data:000000014001AE16 db 0Dh
.data:000000014001AE17 db 0
.data:000000014001AE18 db 0Ch
.data:000000014001AE19 db 0
.data:000000014001AE1A db 15h
.data:000000014001AE1B db 0
.data:000000014001AE1C db 0Ch
.data:000000014001AE1D db 0
.data:000000014001AE1E db 0Bh
.data:000000014001AE1F db 0
.data:000000014001AE20 db 16h
.data:000000014001AE21 db 0
.data:000000014001AE22 db 15h
.data:000000014001AE23 db 0
.data:000000014001AE24 db 14h
.data:000000014001AE25 db 0
.data:000000014001AE26 db 0Dh
.data:000000014001AE27 db 0
.data:000000014001AE28 db 17h
.data:000000014001AE29 db 0
.data:000000014001AE2A db 16h
.data:000000014001AE2B db 0
.data:000000014001AE2C db 0Bh
.data:000000014001AE2D db 0
.data:000000014001AE2E db 0Eh
.data:000000014001AE2F db 0
.data:000000014001AE30 db 0Dh
.data:000000014001AE31 db 0
.data:000000014001AE32 db 14h
.data:000000014001AE33 db 0
.data:000000014001AE34 db 18h
.data:000000014001AE35 db 0
.data:000000014001AE36 db 17h
.data:000000014001AE37 db 0
.data:000000014001AE38 db 0Bh
.data:000000014001AE39 db 0
.data:000000014001AE3A db 14h
.data:000000014001AE3B db 0
.data:000000014001AE3C db 13h
.data:000000014001AE3D db 0
.data:000000014001AE3E db 0Eh

```


Game Reversing

• sub_140001900()에 대한 분석 정리

- background.png를 &ppTexture에 저장 (나중에 background를 그리나 보다...)
- 뭔가 함수를 몇 개 호출하고, 뭔가에 뭔가를 채움...
- 요게 핵심이지 않나 싶음...
정점, 인덱스 정보를 ppBackBuffer에 저장 ➔ 무슨 정점과 인덱스일까..?



ppBackBuffer: 그래픽 프로그래밍에서 화면에 그려질 픽셀 데이터를 임시로 담는 백버퍼 포인터

```
189 (*(&pDestRect + 96164))(pDestRect);
190 v13 = &unk_140020470;
191 face_accum = 0;
192 ppMesh_out = &unk_140020470;
193 table_idx = 0164;
194 cnt = 68164;
195 do
196 {
197     D3DXCreateMeshFVF(
198         *(&index_buffer + table_idx * 2),
199         *table[table_idx],
200         544164,
201         322164,
202         pDevice,
203         ppMesh_out);
204
205     // D:\Users\baekjoon\gcc\mingw64\include\d3dx9mesh.h::ID3DXMesh 선언부 15,16,17,18함수는 Lock,Unlock 함수관련
206     (*(&v13 + 120164))(*v13, 0164, &ppBackBuffer);
207     memcpy(ppBackBuffer, &vertex_layout + 24 * size, 24164 * *table[table_idx]); // 24byte 단위 저장
208     (*(&v13 + 128164))(*v13);
209     (*(&v13 + 136164))(*v13, 0164, &ppBackBuffer);
210     memcpy(ppBackBuffer, &index_layout + 6 * face_accum, 6164 * *(&index_buffer + table_idx * 2)); // 6byte 단위 저장
211     (*(&v13 + 144164))(*v13);
212     size += *table[table_idx];
213     ++v13;
214     face_accum += *(&index_buffer + table_idx * 2); // 면 개수를 누적 저장함.. 왜?
215     table_idx += 2164;
216     ppMesh_out += 8;
217     --cnt;
218 }
219 while ( cnt );
220 }
```

```
do
{
    ppBackBuffer_ += 8;
    v5 = *something_struct_;
    v6 = *(something_struct_ + 1);
    something_struct_ += 32;
    *(ppBackBuffer_ - 8) = v5;
    v7 = *(something_struct_ - 6);
    *(ppBackBuffer_ - 7) = v6;
    v8 = *(something_struct_ - 5);
    *(ppBackBuffer_ - 6) = v7;
    v9 = *(something_struct_ - 4);
    *(ppBackBuffer_ - 5) = v8;
    v10 = *(something_struct_ - 3);
    *(ppBackBuffer_ - 4) = v9;
    v11 = *(something_struct_ - 2);
    *(ppBackBuffer_ - 3) = v10;
    v12 = *(something_struct_ - 1);
    *(ppBackBuffer_ - 2) = v11;
    *(ppBackBuffer_ - 1) = v12;
    --v4;
}
while ( v4 );
```

Game Reversing

- 정말 핵심은? 무한 루프 내부의 함수.

1. `Msg.message == 18`: `WM_QUIT` 메시지를 의미.

사용자가 창을 닫거나, 프로그램에서 `PostQuitMessage(0);`을 호출하면 OS가 발생 시킴

2. 뭔가 구조체 내부의 함수를 짚욱 호출함. 뭐.. 입출력 관련 함수겠지..

3. if 분기가 눈에 띄.

`if v25[1] < 0`, `if v25[30] < 0`, `if v25[32] < 0`

개중에

`*(float *)&dword_140020468 = *(float *)... - 1.0;`

`*(float *)&dword_140020468 = *(float *)... + 1.0;`

동일한 변수에 +1, -1을 수행함.. 수상해 수상해. 미심쩍어 미심쩍어.

뭔까..?

```
while ( 1 )
{
    while ( PeekMessageW(&Msg, 0i64, 0, 0, 1u) )// 대기 시간 없이 실시간 동작을 위한 구조
    {
        TranslateMessage(&Msg);
        DispatchMessageW(&Msg);
    }
    if ( Msg.message == 18 )
        break;

    // 뭔가 함수 호출
    (*(qword_140020460 + 56i64))(qword_140020460);
    (*(qword_140020420 + 56i64))(qword_140020420);
    (*(qword_140020420 + 72i64))(qword_140020420, 16i64, &v23);
    (*(qword_140020460 + 72i64))(qword_140020460, 256i64, v25);

    if ( v25[1] < 0 ) // esc
        PostMessageW(Window, 2u, 0i64, 0i64); // 2u: #define WM_DESTROY 0x0002
    v7 = *dword_140020468;
    if ( v25[30] < 0 )
    {
        v7 = *dword_140020468 - 1.0;
        *dword_140020468 = *dword_140020468 + -1.0;
    }
    if ( v25[32] < 0 )
        *dword_140020468 = v7 + 1.0;
    if ( v23 != xmmword_140020448 )
    {
        *dword_14002046C = *dword_14002046C + (xmmword_140020448 * 0.001);
        xmmword_140020448 = v23;
        *dword_140020690 = *dword_140020690 + (SDWORD1(v23) * 0.001);
    }
    traget_func2(); // 매 프레임 뭔가 그림
```

Game Reversing

- 저 v25의 정체가 궁금함.
- 저 v25가 처음 사용될 때의 흐름을 쫓아가면...
- 이 함수가 ppvOut을 반환 → ppvOut: IDirectInput8 Interface

```
70 DirectInput8Create(hInstance, 0x800u, &riidIIf, &ppvOut, 0i64); // 하드웨어 입력을 받아서 처리(외부함수)
71 (*(*ppvOut + 24i64))(ppvOut, &unk_140003380, &qword_140020460, 0i64);
72 (*(*ppvOut + 24i64))(ppvOut, &unk_140003370, &qword_140020420, 0i64);
73 (*(*qword_140020460 + 88i64))(qword_140020460, &unk_1400032E0);
74 (*(*qword_140020460 + 104i64))(qword_140020460, Window, 6i64);
75 (*(*qword_140020420 + 88i64))(qword_140020420, &unk_140003300);
76 (*(*qword_140020420 + 104i64))(qword_140020420, Window, 21i64);
77 while ( 1 )
78 {
79     while ( PeekMessageW(&Msg, 0i64, 0, 0, 1u) )// 대기 시간 없이 실시간 동작을 위한 구조
80     {
81         TranslateMessage(&Msg);
82         DispatchMessageW(&Msg);
83     }
84     if ( Msg.message == 18 )
85         break;
86
87
88     // 뭔가 함수 호출
89     (*(*qword_140020460 + 56i64))(qword_140020460);
90     (*(*qword_140020420 + 56i64))(qword_140020420);
91     (*(*qword_140020420 + 72i64))(qword_140020420, 16i64, &v23);
92     (*(*qword_140020460 + 72i64))(qword_140020460, 256i64, v25);
```

DirectInput8 Interface

DirectInput8

Summarize this article for me

In this article

IDirectInput8 Members

Remarks

Summarize this

Expand table

Copy

Game Reverse

- $(*(\text{ppvOut} + 24 \times 64))(\text{ppvOut}, \&\text{unk_140003380}, \&\text{qword_140020460}, 25)$
- $\text{ppvOut} + 24$: CreateDevice 함수 위치
→ 키보드 장치 객체(IDirectInputDevice8) 를 qword_140020460 에 저장
- $(*(\text{qword_140020460} + 72 \times 64))(\text{qword_140020460}, 25)$
- $\text{qword_140020460} + 72$: GetDeviceState 함수 위치
→ v25: 현재 눌린 키 상태(0 or 1)가 기록되는 256byte 크기 배열

```

70  IDirectInput8Create(hInstance, 0x800u, &riidltf,
71  (*(ppvOut + 24i64))(ppvOut, &unk_140003380, &qword_140020460, 25),
72  (*(ppvOut + 24i64))(ppvOut, &unk_140003370, &qword_140020460, 25),
73  (*(qword_140020460 + 88i64))(qword_140020460, &unk_140020460, 25),
74  (*(qword_140020460 + 104i64))(qword_140020460, &unk_140020460, 25),
75  (*(qword_140020420 + 88i64))(qword_140020420, &unk_140020420, 25),
76  (*(qword_140020420 + 104i64))(qword_140020420, &unk_140020420, 25),
77  while ( 1 )
78  {
79      while ( PeekMessageW(&Msg, 0i64, 0, 0, 1u) )//
80      {
81          TranslateMessage(&Msg);
82          DispatchMessageW(&Msg);
83      }
84      if ( Msg.message == 18 )
85          break;
86
87
88      // 뭔가 함수 호출
89      (*(qword_140020460 + 56i64))(qword_140020460, &unk_140020460, 25),
90      (*(qword_140020420 + 56i64))(qword_140020420, &unk_140020420, 25),
91      (*(qword_140020420 + 72i64))(qword_140020420, &unk_140020420, 25),
92      (*(qword_140020460 + 72i64))(qword_140020460, &unk_140020460, 25),

```

IDirectInputDevice8 Members

Expand table

Method	Description
IDirectInputDevice8::Acquire	Obtains access to the input device.
IDirectInputDevice8::BuildActionMap	Builds an action map for the device and retrieves information about it.
IDirectInputDevice8::CreateEffect	Creates and initializes an instance of an effect identified by the effect globally unique identifier (GUID).
IDirectInputDevice8::EnumCreatedEffectObjects	Enumerates all the currently created effects for this device.
IDirectInputDevice8::EnumEffects	Enumerates all the effects supported by the force-feedback system on the device.
IDirectInputDevice8::EnumEffectsInFile	Enumerates all the effects in a file created by the Force Editor utility or another application using the same file format.
IDirectInputDevice8::EnumObjects	Enumerates the input and output objects available on a device.
IDirectInputDevice8::Escape	Sends a hardware-specific command to the force-feedback driver.
IDirectInputDevice8::GetCapabilities	Obtains the capabilities of the DirectInputDevice object.
IDirectInputDevice8::GetDeviceData	Retrieves buffered data from the device.
IDirectInputDevice8::GetDeviceInfo	Obtains information about the device's identity.
IDirectInputDevice8::GetDeviceState	Retrieves immediate data from the device.
IDirectInputDevice8::GetEffectInfo	Obtains information about an effect.
IDirectInputDevice8::GetForceFeedbackState	Retrieves the state of the device's force-feedback system.
IDirectInputDevice8::GetImageInfo	Retrieves information about a device image for use in

Game Reversing

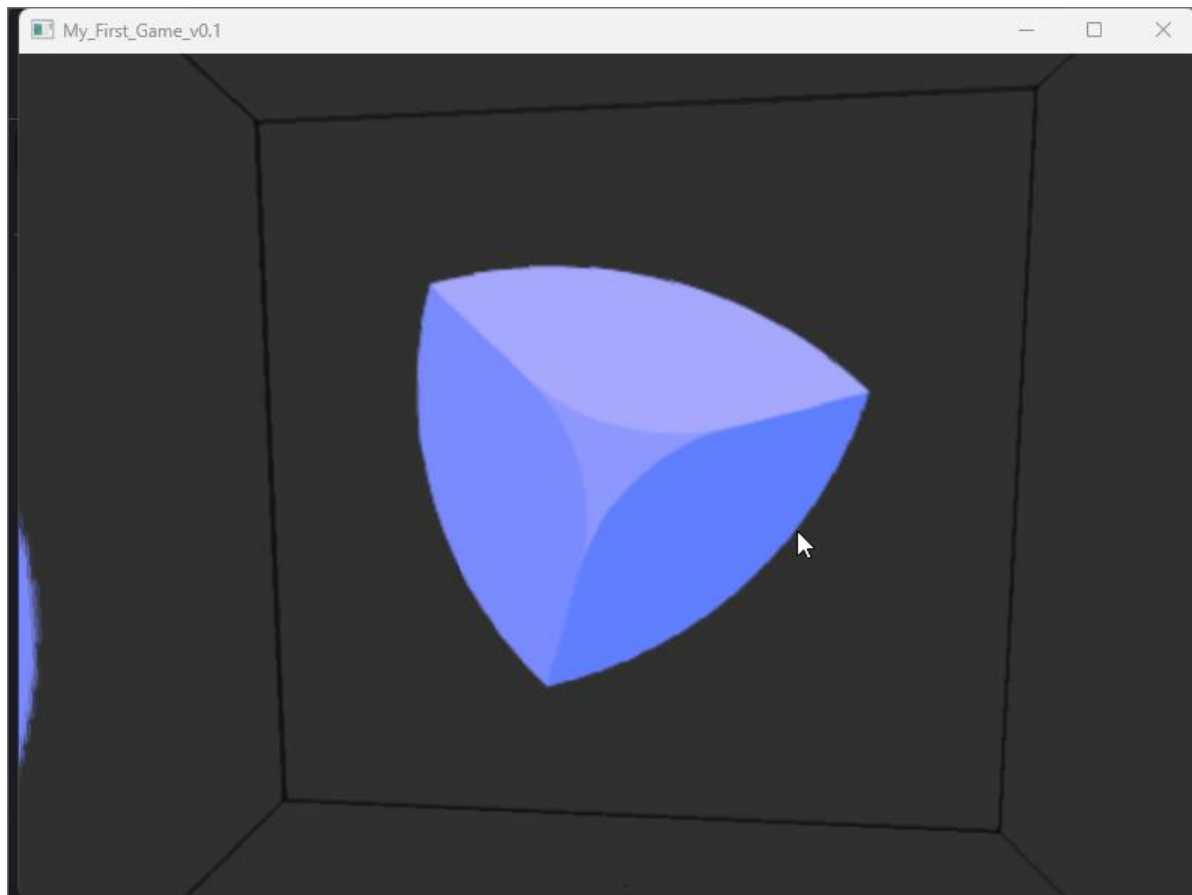
- 그림
- v25[30]: 0x1E: DIK_A (A키)
- v25[32]: 0x20: DIK_D (D키)
- PoC

```
while ( 1 )
{
    while ( PeekMessageW(&Msg, 0i64, 0, 0, 1u) )// 대기 시간 없이 실시간 동작을 위한 구조
    {
        TranslateMessage(&Msg);
        DispatchMessageW(&Msg);
    }
    if ( Msg.message == 18 )
        break;

    // 뭔가 함수 호출
    (*(&qword_140020460 + 56i64))(qword_140020460);
    (*(&qword_140020420 + 56i64))(qword_140020420);
    (*(&qword_140020420 + 72i64))(qword_140020420, 16i64, &v23);
    (*(&qword_140020460 + 72i64))(qword_140020460, 256i64, v25);

    if ( v25[1] < 0 ) // esc
        PostMessageW(Window, 2u, 0i64, 0i64); // 2u: #define WM_DESTROY 0x0002
    v7 = &dword_140020468;
    if ( v25[30] < 0 ) // 30: 0x1E: DIK_A (A키)
    {
        v7 = &dword_140020468 + -1.0;
        &dword_140020468 = &dword_140020468 + -1.0;
    }
    if ( v25[32] < 0 ) // 32: 0x20: DIK_D (D키)
        &dword_140020468 = v7 + 1.0;
    if ( v23 != xmmword_140020448 )
    {
        &dword_14002046C = &dword_14002046C + (xmmword_140020448 * 0.001);
        xmmword_140020448 = v23;
        &dword_140020690 = &dword_140020690 + (SDWORD1(v23) * 0.001);
    }
    traget_func2(); // 매 프레임 뭔가 그림
}
```

Poc. 1



Game Reversing

- A, D 키만 작동 가능하고 벽에 닿으면 그 이상의 좌표로 넘어가지 않음.
 - 일단 최종 목표인 FLAG의 실마리를 찾음.
 - 맵을 벗어나는 행위 자체를 막는 코드가 반드시 있을 것임.
 - 그 코드는 아마도 무한 루프 내부에 있어야 가능할 것임.
 - 그 코드를 어떻게 찾아서 수정하던가. 좌표 값을 실시간 수정하면 보일 듯 함.
 - 어케 하노...

1. Patch Program이라고 명령어 자체를 패치하는 방법

2. 실시간 좌표 값의 위치를 찾아 수정하는 방법 ☒

3. 치트 엔진?

```
109 while ( 1 ) // 무한 루프
110 {
111     while ( PeekMessageW(&Msg, 0i64, 0, 0, 1u) )
112     {
113         TranslateMessage(&Msg);
114         DispatchMessageW(&Msg);
115     }
116     if ( Msg.message == 18 )
117         break;
118     (*(void (__fastcall *))(__int64))(*(_QWORD *)qword_140020460 + 56i64);
119     (*(void (__fastcall *))(__int64))(*(_QWORD *)qword_140020420 + 56i64);
120     (*(void (__fastcall *))(__int64, __int64, __int128 *))(*(_QWORD *)qword_140020420,
121     16i64,
122     &v23);
123     (*(void (__fastcall *))(__int64, __int64, char *))(*(_QWORD *)qword_140020460,
124     256i64,
125     v25);
126     if ( v25[1] < 0 )
127     {
128         PostMessageW(Window, 2u, 0i64, 0i64);
129         v7 = *(float *)&dword_140020468;
130         if ( v25[30] < 0 )
131         {
132             v7 = *(float *)&dword_140020468 + -1.0;
133             *(float *)&dword_140020468 = *(float *)&dword_140020468 + -1.0;
134         }
135         if ( v25[32] < 0 )
136             *(float *)&dword_140020468 = v7 + 1.0;
137         if ( (_QWORD)v23 != (_QWORD)xmmword_140020448 )
138         {
139             *(float *)&dword_14002046C = *(float *)&dword_14002046C + (float)xmmword_140020448 - v23;
140             xmmword_140020448 = v23;
141             *(float *)&dword_140020690 = *(float *)&dword_140020690 + (float)xmmword_140020448 - v23;
142         }
143     }
144     sub_140001520();
145 }
```

Game Reversing

- 원가 -300.0보다 작거나,
- 원가 300보다 크면
- float 자료형 기준: -299.0, 299.0으로 원가 값을 초기화 함.
- 처음 했던 생각: 마우스를 위 아래로 드래그 하면 배경 객체가 뒤집히는 현상이 발생했는데 이게 그건가 싶었음.
- 근데 벽을 뚫을 수 없는 코드가 저거밖에 안 보임.
- i.e.) 벽을 뚫을 수 없는 코드가 저거임.
- 그럼 무슨 시도를 해야 하나?
- 벽에 딱 붙어서 값이 300.0 이거나, 299.0인 놈을 찾으면
- 그 놈이 좌표값임.
- 그럼 저 변수명이 dword_140020694인데
자료형 뒤의 값은 저 값의 주소를 의미함.
- 저 데이터가 저장되어 있는 영역의 시작 주소 + 0x20694를
보자.

Name	Start	End	R	W
.text	0000000140001000	0000000140003000	R	.
.idata	0000000140003000	0000000140003250	R	.
.rdata	0000000140003250	0000000140005000	R	.
.data	0000000140005000	0000000140021000	R	W
.pdata	0000000140021000	0000000140022000	R	.

```

Pseudocode-A      Stack of WinMain      IDA View-A      Hex
21  (*pDevice + 712i64))(pDevice, 322i64);
22  D3DXMatrixRotationYawPitchRoll(pMatrix);           // yaw(가로),pitch(세로),roll(회전) 축을 기준으로 물체를 회전
23  v0 = *dword_140020694 + *dword_140020468;
24  dword_140020694 = *dword_140020694 + *dword_140020468;
25  if ( *dword_140020694 < 300.0 )                     // 원가 시점이 오버되면 초기화?
26  {
27      if ( v0 ≤ -300.0 )
28          dword_140020694 = -1013612544;
29  }
30  else
31  {
32      dword_140020694 = 1133871104;
33  }
34  dword_140020468 = 0;
35  pVectorOut = 0i64;
36  v7 = 1.0;
37  *pVector = 0i64;
38  *spVector[8] = 1065353216;
39  D3DXVec3TransformCoord(&pVectorOut, pVector, pMatrix);
40  v8[0] = dword_140020694;
41  v8[2] = dword_140020698;
42  *spVectorOut = *spVectorOut + *dword_140020694;
43  v8[1] = 1095027917;
44  *(&pVectorOut + 1) = *(&pVectorOut + 1) + 12.3;
45  *pVector = 0;
46  *spVector[4] = 1065353216i64;
47  v7 = v7 + *dword_140020698;
48  D3DXMatrixLookAtLH(v12, v8, &pVectorOut, pVector);
49  (*pDevice + 352i64))(pDevice, 2i64, v12);
50  D3DXMatrixPerspectiveFovLH(v13);
51  (*pDevice + 352i64))(pDevice, 3i64, v13);
52  D3DXMatrixScaling(v14);
53  v1 = 0;
54  v2 = &unk_140020470;
55  do
56  {
57      D3DXMatrixTranslation(v10);
58      D3DXMatrixMultiply(v5, v10, v14);
59      v9[0] = v5[0];
60      v9[1] = v5[1];
61      v9[2] = v5[2];
62      v9[3] = v5[3];
63      (*pDevice + 352i64))(pDevice, 256i64, v9);
64      (*v2 + 24i64))(*v2, 0i64);
65      ++v1;
66      ++v2;
67  }
68  while ( v1 < 68 );
69  (*pDevice + 800i64))(pDevice, 0i64, pDestRect, 0i64, 24);
70  (*pDevice + 520i64))(pDevice, 0i64, ppTexture); // ppTexture: target_func1에서 본 background.png가 저장되어 있음
71  D3DXMatrixTranslation(v10);
72  (*pDevice + 352i64))(pDevice, 256i64, v10);
    
```


WinDbg: edit

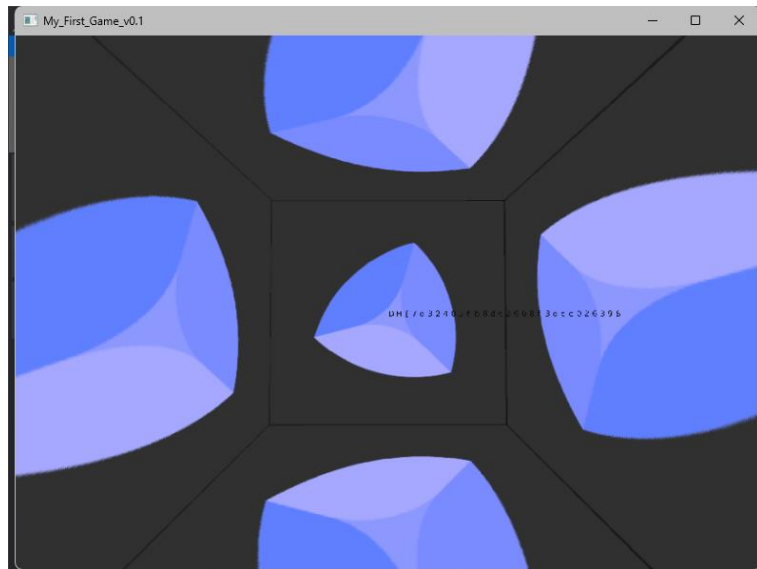
- MS에서 지원하는 Debug 도구

```
0:027> lm
start          end             module_name
00007ff6`fb230000 00007ff6`fb26b000 WindowsProject1 C (no symbols)
00007ff9`543d0000 00007ff9`54633000 d3dx9_43 (deferred)
00007ff9`58490000 00007ff9`59b9e000 igd9dxva64 (deferred)
```

```
0:027> dd 00007ff6`fb250694 L2
00007ff6`fb250694  43958000 00000000
0:027> dx *(float*)0x00007ff6`fb250694
*(float*)0x00007ff6`fb250694 : 299.000000 [Type: float]
```

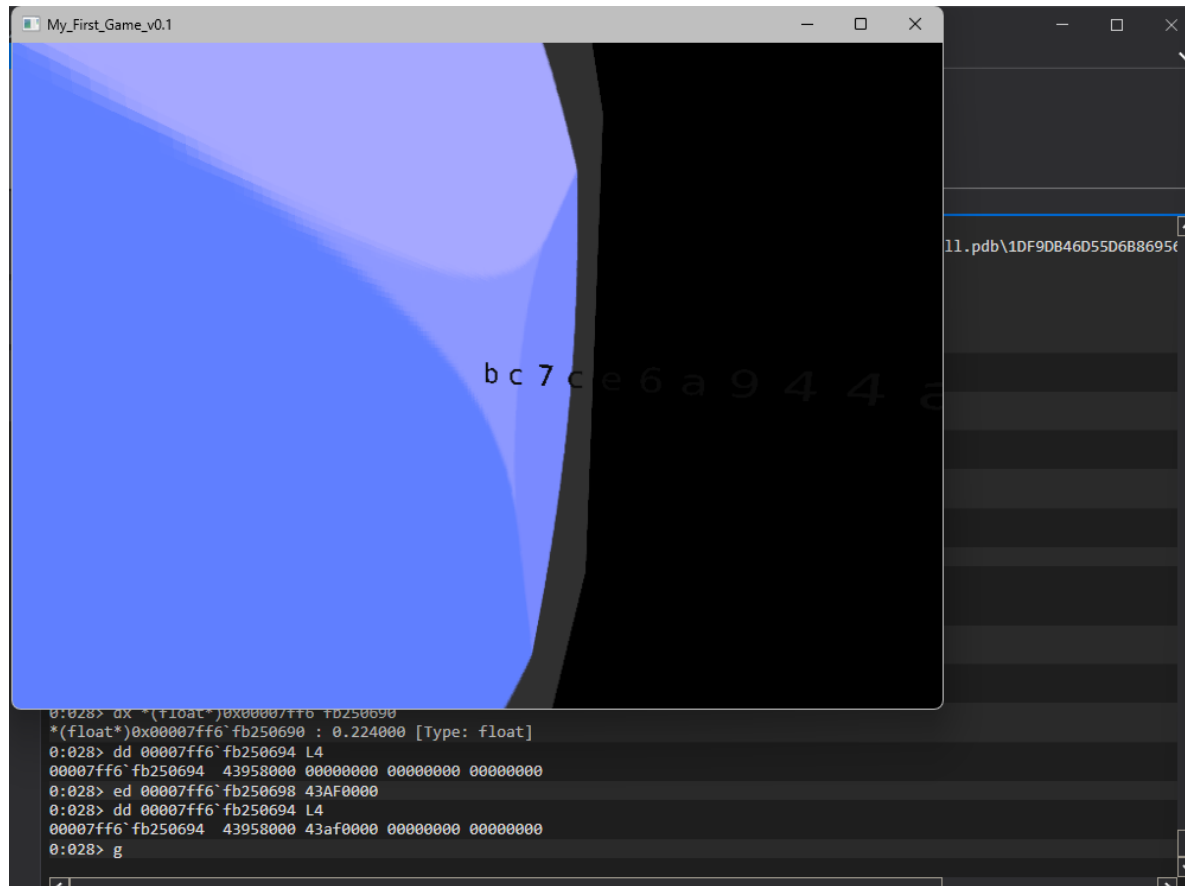
- D를 꺾눌러 벽에 닿았을 때: 300.0에 도달해 299.0으로 초기화 됨을 확인
- 그럼, 난 오른쪽으로 가고 싶는데 어떻게 해야할까..?
- 전제를 두자. 좌표는 구조체로 관리된다는 전제
- 그럼 구조체로써 뭉쳐있겠지..?

```
0:028> dd 00007ff6`fb250690 L4
00007ff6`fb250690  3e65604a 43958000 00000000 00000000
```



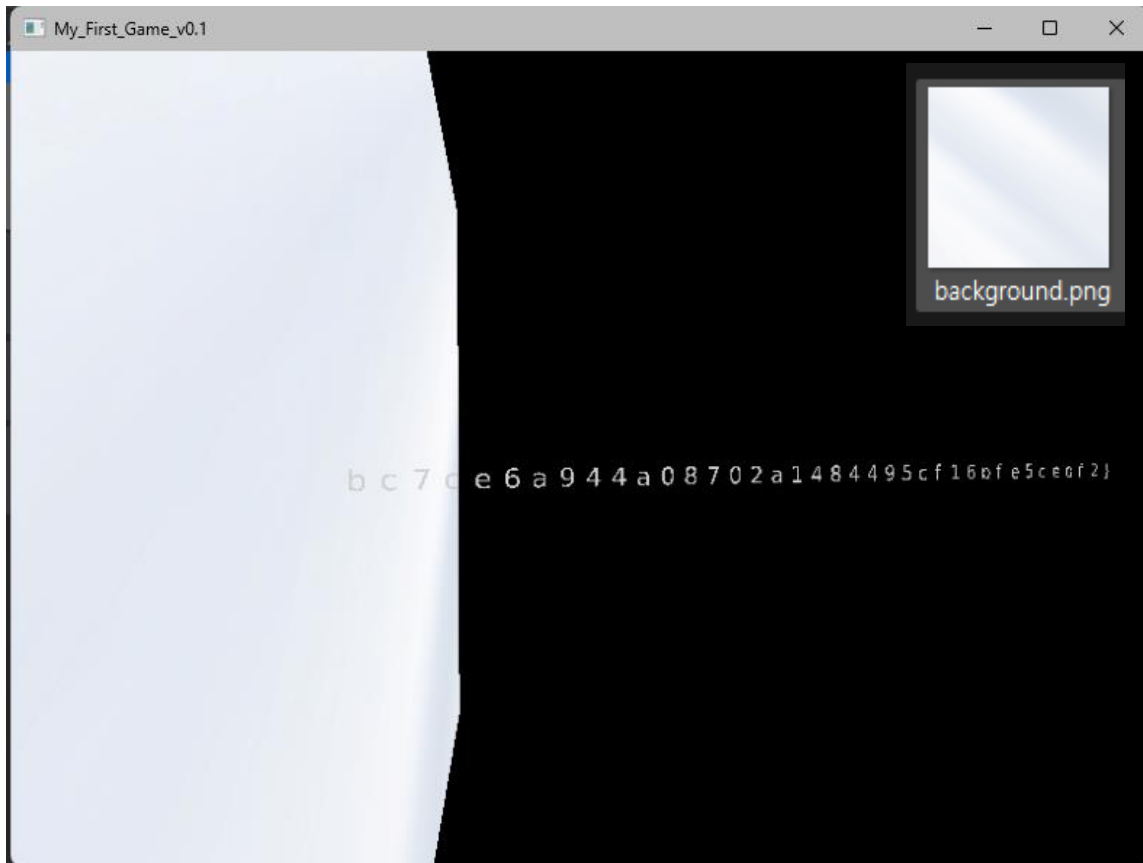
PoC. 2

- 해당 .exe가 로드된 시작주소에서
- +0x20694 주소에서
- 뒤를 읽어보니 모두 0임.
- 그럼.. 좌표를 구조체로 조정한다는 전제로
- 바로 뒤의 4바이트 값을 수정
→ ed 00..694+4 43AF0000
→ 43AF0000: IEEE 방식으로 350.0
- 맵을 뚫고 플래그가 마저 보이는 모습 확인
- 배경을 하얀색으로 변경할 필요가 있다 판단.



PoC. 2

- 배경.png 수정 후
- 마찬가지로 좌표 위치의 값을 350.0으로 수정
→ 가로 좌표가 맵을 뚫고 나감.
- 나머지 플래그 확인



Thank you